

```
In [1]: !pip install torch torchvision torchaudio scikit-learn opencv-python
```

```
Collecting torch
  Downloading torch-2.7.1-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (29 kB)
Collecting torchvision
  Downloading torchvision-0.22.1-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (6.1 kB)
Collecting torchaudio
  Downloading torchaudio-2.7.1-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.7.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (11 kB)
Collecting opencv-python
  Downloading opencv_python-4.12.0.88-cp37-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (19 kB)
Collecting filelock (from torch)
  Downloading filelock-3.18.0-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in /opt/conda/lib/python3.12/site-packages (from torch) (4.14.1)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-packages (from torch) (80.1.0)
Collecting sympy>=1.13.3 (from torch)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx (from torch)
  Downloading networkx-3.5-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: Jinja2 in /opt/conda/lib/python3.12/site-packages (from torch) (3.1.6)
Collecting fsspec (from torch)
  Downloading fsspec-2025.7.0-py3-none-any.whl.metadata (12 kB)
Collecting nvidia-cuda-nvrtc-cu12==12.6.77 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.6.77-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.6.77 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.6.77-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.6.80 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.6.80-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.5.1.17 (from torch)
  Downloading nvidia_cudnn_cu12-9.5.1.17-py3-none-manylinux_2_28_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.6.4.1 (from torch)
  Downloading nvidia_cublas_cu12-12.6.4.1-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.3.0.4 (from torch)
  Downloading nvidia_cufft_cu12-11.3.0.4-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.7.77 (from torch)
  Downloading nvidia_curand_cu12-10.3.7.77-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.7.1.2 (from torch)
  Downloading nvidia_cusolver_cu12-11.7.1.2-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparselt-cu12==12.5.4.2 (from torch)
  Downloading nvidia_cusparselt_cu12-12.5.4.2-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparselt-cu12==0.6.3 (from torch)
  Downloading nvidia_cusparselt_cu12-0.6.3-py3-none-manylinux2014_x86_64.whl.metadata (6.8 kB)
Collecting nvidia-nccl-cu12==2.26.2 (from torch)
  Downloading nvidia_nccl_cu12-2.26.2-py3-none-manylinux2014_x86_64.manylinux_2_1
```

```

7_x86_64.whl.metadata (2.0 kB)
Collecting nvidia-nvtx-cu12==12.6.77 (from torch)
  Downloading nvidia_nvtx_cu12-12.6.77-py3-none-manylinux2014_x86_64.manylinux_2_
17_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-nvjitlink-cu12==12.6.85 (from torch)
  Downloading nvidia_nvjitlink_cu12-12.6.85-py3-none-manylinux2010_x86_64.manylin
ux_2_12_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufile-cu12==1.11.1.6 (from torch)
  Downloading nvidia_cufile_cu12-1.11.1.6-py3-none-manylinux2014_x86_64.manylinux
_2_17_x86_64.whl.metadata (1.5 kB)
Collecting triton==3.3.1 (from torch)
  Downloading triton-3.3.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_6
4.whl.metadata (1.5 kB)
Requirement already satisfied: numpy in /opt/conda/lib/python3.12/site-packages
(from torchvision) (2.3.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.1
2/site-packages (from torchvision) (11.3.0)
Collecting scipy>=1.8.0 (from scikit-learn)
  Downloading scipy-1.16.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_6
4.whl.metadata (61 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.1-py3-none-any.whl.metadata (5.6 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting numpy (from torchvision)
  Downloading numpy-2.2.6-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.
whl.metadata (62 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.12/site-
packages (from jinja2->torch) (3.0.2)
Downloading torch-2.7.1-cp312-cp312-manylinux_2_28_x86_64.whl (821.0 MB)
_____ 821.0/821.0 MB 94.9 MB/s eta 0:00:00
0:0100:01
Downloading nvidia_cublas_cu12-12.6.4.1-py3-none-manylinux2014_x86_64.manylinux_2
_17_x86_64.whl (393.1 MB)
_____ 393.1/393.1 MB 165.7 MB/s eta 0:00:00
0:0100:01
Downloading nvidia_cuda_cupti_cu12-12.6.80-py3-none-manylinux2014_x86_64.manylinu
x_2_17_x86_64.whl (8.9 MB)
_____ 8.9/8.9 MB 246.5 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.6.77-py3-none-manylinux2014_x86_64.whl (23.
7 MB)
_____ 23.7/23.7 MB 250.0 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.6.77-py3-none-manylinux2014_x86_64.manyli
nux_2_17_x86_64.whl (897 kB)
_____ 897.7/897.7 kB 77.1 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.5.1.17-py3-none-manylinux_2_28_x86_64.whl (571.0
MB)
_____ 571.0/571.0 MB 161.0 MB/s eta 0:00:00
0:0100:01
Downloading nvidia_cufft_cu12-11.3.0.4-py3-none-manylinux2014_x86_64.manylinux_2_
17_x86_64.whl (200.2 MB)
_____ 200.2/200.2 MB 226.0 MB/s eta 0:00:00
0:0100:01
Downloading nvidia_cufile_cu12-1.11.1.6-py3-none-manylinux2014_x86_64.manylinux_2
_17_x86_64.whl (1.1 MB)
_____ 1.1/1.1 MB 53.0 MB/s eta 0:00:00
Downloading nvidia_curand_cu12-10.3.7.77-py3-none-manylinux2014_x86_64.manylinux_
2_17_x86_64.whl (56.3 MB)

```

```

_____ 56.3/56.3 MB 158.1 MB/s eta 0:00:00a
0:00:01
Downloading nvidia_cusolver_cu12-11.7.1.2-py3-none-manylinux2014_x86_64.manylinux
_2_17_x86_64.whl (158.2 MB)
_____ 158.2/158.2 MB 227.7 MB/s eta 0:00:000
0:0100:01
Downloading nvidia_cusparses_cu12-12.5.4.2-py3-none-manylinux2014_x86_64.manylinux
_2_17_x86_64.whl (216.6 MB)
_____ 216.6/216.6 MB 245.6 MB/s eta 0:00:000
0:0100:01
Downloading nvidia_cusparselt_cu12-0.6.3-py3-none-manylinux2014_x86_64.whl (156.8
MB)
_____ 156.8/156.8 MB 218.1 MB/s eta 0:00:000
0:0100:01
Downloading nvidia_nccl_cu12-2.26.2-py3-none-manylinux2014_x86_64.manylinux_2_17_
x86_64.whl (201.3 MB)
_____ 201.3/201.3 MB 253.0 MB/s eta 0:00:000
0:0100:01
Downloading nvidia_nvjitlink_cu12-12.6.85-py3-none-manylinux2010_x86_64.manylinux
_2_12_x86_64.whl (19.7 MB)
_____ 19.7/19.7 MB 179.9 MB/s eta 0:00:00
Downloading nvidia_nvtx_cu12-12.6.77-py3-none-manylinux2014_x86_64.manylinux_2_17
_x86_64.whl (89 kB)
Downloading triton-3.3.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.
whl (155.7 MB)
_____ 155.7/155.7 MB 146.1 MB/s eta 0:00:000
0:0100:01
Downloading torchvision-0.22.1-cp312-cp312-manylinux_2_28_x86_64.whl (7.5 MB)
_____ 7.5/7.5 MB 169.2 MB/s eta 0:00:00
Downloading torchaudio-2.7.1-cp312-cp312-manylinux_2_28_x86_64.whl (3.5 MB)
_____ 3.5/3.5 MB 201.4 MB/s eta 0:00:00
Downloading scikit_learn-1.7.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x8
6_64.whl (9.5 MB)
_____ 9.5/9.5 MB 225.3 MB/s eta 0:00:00
Downloading opencv_python-4.12.0.88-cp37-abi3-manylinux2014_x86_64.manylinux_2_17
_x86_64.whl (67.0 MB)
_____ 67.0/67.0 MB 272.4 MB/s eta 0:00:000
0:01
Downloading numpy-2.2.6-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.wh
l (16.5 MB)
_____ 16.5/16.5 MB 213.5 MB/s eta 0:00:00
Downloading joblib-1.5.1-py3-none-any.whl (307 kB)
Downloading scipy-1.16.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.w
hl (35.2 MB)
_____ 35.2/35.2 MB 98.2 MB/s eta 0:00:00 0:
00:01
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
_____ 6.3/6.3 MB 180.9 MB/s eta 0:00:00
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
_____ 536.2/536.2 kB 68.8 MB/s eta 0:00:00
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Downloading filelock-3.18.0-py3-none-any.whl (16 kB)
Downloading fsspec-2025.7.0-py3-none-any.whl (199 kB)
Downloading networkx-3.5-py3-none-any.whl (2.0 MB)
_____ 2.0/2.0 MB 225.4 MB/s eta 0:00:00
Installing collected packages: nvidia-cusparselt-cu12, mpmath, triton, threadpool
ctl, sympy, nvidia-nvtx-cu12, nvidia-nvjitlink-cu12, nvidia-nccl-cu12, nvidia-cur
and-cu12, nvidia-cufile-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, n
vidia-cuda-cupti-cu12, nvidia-cublas-cu12, numpy, networkx, joblib, fsspec, filel
ock, scipy, opencv-python, nvidia-cusparses-cu12, nvidia-cufft-cu12, nvidia-cudnn-

```

```

cu12, scikit-learn, nvidia-cusolver-cu12, torch, torchvision, torchaudio
Attempting uninstall: numpy----- 13/29 [nvidia-cublas-cu12]u1
2]
Found existing installation: numpy 2.3.2----- 13/29 [nvidia-cubl
as-cu12]
Uninstalling numpy-2.3.2:m----- 13/29 [nvidia-cublas-cu12]
Successfully uninstalled numpy-2.3.2[90m----- 14/29 [numpy]l
as-cu12]
----- 29/29 [torchaudio]9 [torchaudio]]ver-
cu12]
ERROR: pip's dependency resolver does not currently take into account all the pac
kages that are installed. This behaviour is the source of the following dependenc
y conflicts.
pyspark-client 4.0.0 requires googleapis-common-protos>=1.65.0, which is not inst
alled.
pyspark-client 4.0.0 requires grpcio>=1.67.0, which is not installed.
pyspark-client 4.0.0 requires grpcio-status>=1.67.0, which is not installed.
pyspark-client 4.0.0 requires pyarrow>=11.0.0, which is not installed.
pyspark-connect 4.0.0 requires googleapis-common-protos>=1.65.0, which is not ins
talled.
pyspark-connect 4.0.0 requires grpcio>=1.67.0, which is not installed.
pyspark-connect 4.0.0 requires grpcio-status>=1.67.0, which is not installed.
pyspark-connect 4.0.0 requires pyarrow>=11.0.0, which is not installed.
Successfully installed filelock-3.18.0 fsspec-2025.7.0 joblib-1.5.1 mpmath-1.3.0
networkx-3.5 numpy-2.2.6 nvidia-cublas-cu12-12.6.4.1 nvidia-cuda-cupti-cu12-12.6.
80 nvidia-cuda-nvrtc-cu12-12.6.77 nvidia-cuda-runtime-cu12-12.6.77 nvidia-cudnn-c
u12-9.5.1.17 nvidia-cufft-cu12-11.3.0.4 nvidia-cufile-cu12-1.11.1.6 nvidia-curand
-cu12-10.3.7.77 nvidia-cusolver-cu12-11.7.1.2 nvidia-cusparselt-cu12-0.6.3 nvidia-nccl-cu12-2.26.2 nvidia-nvjitlink-cu12-12.6.85 nvi
dia-nvtx-cu12-12.6.77 opencv-python-4.12.0.88 scikit-learn-1.7.1 scipy-1.16.1 sym
py-1.14.0 threadpoolctl-3.6.0 torch-2.7.1 torchaudio-2.7.1 torchvision-0.22.1 tri
ton-3.3.1

```

In [7]:

```

"""
Created on Aug 04 2025

@author: vigo
"""

import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision.utils import make_grid
import matplotlib.pyplot as plt

import numpy as np
import cv2 as cv

import re
import glob

from unet import UNet

torch.cuda.empty_cache()

NET_INPUT_SIZE = (572, 572)
NET_OUTPUT_SIZE = (388, 388)

def Normalize(img):
    return np.array(img/255.0, dtype=np.float32)

```

```

def Shape_Process(img):
    if img.shape > NET_INPUT_SIZE:
        img = cv.resize(img, NET_INPUT_SIZE, interpolation = cv.INTER_CUBIC) #

    elif img.shape < NET_INPUT_SIZE:
        if img.shape[0] % 2:
            img = cv.resize(img, img.shape[0] // 2, interpolation = cv.INTER_CUBIC)
        padd = int((NET_INPUT_SIZE[0] - img.shape[0]) / 2)
        img = cv.copyMakeBorder(img, padd, padd, padd, padd, cv.BORDER_REFLECT_101)

    return img

def Center_Crop(img):
    img = Shape_Process(img)
    crop_yx = (int((img.shape[0] - NET_OUTPUT_SIZE[0]) / 2), int((img.shape[1] - NET_OUTPUT_SIZE[1]) / 2))
    img = img[crop_yx[0]: crop_yx[0] + NET_OUTPUT_SIZE[0], crop_yx[1]: crop_yx[1] + NET_OUTPUT_SIZE[1]]

    return img

def Numerical_Sort(value):
    numbers = re.compile(r'(\d+)')
    parts = numbers.split(value)
    parts[1::2] = map(int, parts[1::2])
    return parts

def Read_Images_Test(folder_path, image_names):
    images = []
    for filename in sorted(glob.glob(folder_path), key=Numerical_Sort):
        image_names.append(filename.split('/')[-1])
        img = cv.imread(filename, flags = cv.IMREAD_GRAYSCALE)
        img = Shape_Process(img)
        images.append(Normalize(img))

    images = np.asarray(images, dtype=np.float32)
    images = images[:, :, :, np.newaxis, np.newaxis] # format => [N, H, W, C1]
    return images

def Read_Images_Train(folder_path, image_names):
    images = []
    for filename in sorted(glob.glob(folder_path), key=Numerical_Sort):
        image_names.append(filename.split('/')[-1])
        img = cv.imread(filename, flags = cv.IMREAD_GRAYSCALE)
        img = Shape_Process(img)
        images.append(Normalize(img))

    images = np.asarray(images, dtype=np.float32)
    images = images[:, :, :, np.newaxis] # format => [N, H, W, C]
    return images

def Read_Images_GTruth(folder_path, image_names):
    labels = []
    for filename in sorted(glob.glob(folder_path), key=Numerical_Sort):
        image_names.append(filename.split('/')[-1])
        mask = cv.imread(filename, flags = cv.IMREAD_GRAYSCALE)
        mask = Center_Crop(mask)
        mask = Normalize(mask)
        mask[mask > 0.5] = 1
        mask[mask <= 0.5] = 0
        labels.append(mask)

```

```

labels = np.asarray(labels, dtype=np.float32)
labels = labels[:, :, :, np.newaxis]    # format => [N, H, W, C]
return labels

def Matplotliblib_Imshow(img):
    npimg = img.cpu().detach().numpy()
    npimg = (npimg * 255.0).astype("int")
    plt.imshow(np.transpose(npimg, (1, 2, 0)))

### dataset load and preprocessing

drive_path = "./data"

test_images_path = drive_path + "/test/*.tif"

train_gtruth_path = drive_path + "/train/labels/*.tif"
train_images_path = drive_path + "/train/images/*.tif"

test_images = []
image_names = []
train_gtruth = []
train_images = []
test_images = Read_Images_Test(test_images_path, image_names)
train_images = Read_Images_Train(train_images_path, image_names)
train_gtruth = Read_Images_GTruth(train_gtruth_path, image_names)

### Create data Loaders.

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Using {device} device')

# pytorch input format => [N, H, W, C1] -> [N, C1, H, W]
train_tensor = torch.from_numpy(train_images).transpose_(2, 3).transpose_(1, 2)
train_gtruth_tensor = torch.from_numpy(train_gtruth).transpose_(2, 3).transpose_(1, 2)
# pytorch input format => [N, H, W, C1, C2] -> [N, C1, C2, H, W]
test_tensor = torch.from_numpy(test_images).transpose_(1, 3).transpose_(2, 4).

batch_size = 2
train_dl = DataLoader(train_tensor, batch_size=batch_size, shuffle=False)
train_gtruth_dl = DataLoader(train_gtruth_tensor, batch_size=batch_size, shuffle=False)
test_dl = DataLoader(test_tensor, batch_size = len(test_images), shuffle=False)
test_iter = next(iter(test_dl))

### create model
model = UNet().to(device)
print(model)
optim = torch.optim.Adam(model.parameters(), lr=0.0001)
n_epochs = 300

### train
for e in range(epoch):
    for i in range(len(train_dl)):
        train_iter = next(iter(train_dl))
        train_gtruth_iter = next(iter(train_gtruth_dl))
        optim.zero_grad()
        prediction = model(train_iter)
        loss = F.mse_loss(prediction, train_gtruth_iter) # Try BCE, DICE Loss
        loss.backward()
        optim.step()

```

```
print (f'Epoch--[{e+1}]/{n_epochs}], Loss: {loss.item()}')

print('Finished Training')

### predict
test_img_index = 0
test_prediction = model(test_iter[test_img_index])

# show images
img_grid = make_grid(test_prediction)
Matplotlib_Imshow(img_grid)

### model save
torch.save(model.state_dict(), "./model.pth")
print("Saved PyTorch Model State to model.pth")

### model Load
model_test = UNet().to(device)
model_test.load_state_dict(torch.load("./model.pth"))

test_prediction = model_test(test_iter[0])
img_grid = make_grid(test_prediction)

# show images
Matplotlib_Imshow(img_grid)
```

Using cuda device

```
UNet(
  (maxpool): MaxPool2d(kernel_size=(2, 2), stride=(2, 2), padding=0, dilation=1,
  ceil_mode=False)
  (drop): Dropout2d(p=0.5, inplace=False)
  (conv1): Double_Conv(
    (double_conv): Sequential(
      (0): Conv2d(1, 64, kernel_size=(3, 3), stride=(1, 1))
      (1): ReLU(inplace=True)
      (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
      (3): ReLU(inplace=True)
    )
  )
  (conv2): Double_Conv(
    (double_conv): Sequential(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1))
      (1): ReLU(inplace=True)
      (2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1))
      (3): ReLU(inplace=True)
    )
  )
  (conv3): Double_Conv(
    (double_conv): Sequential(
      (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1))
      (1): ReLU(inplace=True)
      (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1))
      (3): ReLU(inplace=True)
    )
  )
  (conv4): Double_Conv(
    (double_conv): Sequential(
      (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1))
      (1): ReLU(inplace=True)
      (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1))
      (3): ReLU(inplace=True)
    )
  )
  (conv5): Double_Conv(
    (double_conv): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(3, 3), stride=(1, 1))
      (1): ReLU(inplace=True)
      (2): Conv2d(1024, 1024, kernel_size=(3, 3), stride=(1, 1))
      (3): ReLU(inplace=True)
    )
  )
  (up_conv1): Up_Conv(
    (up_conv): Sequential(
      (0): ConvTranspose2d(1024, 512, kernel_size=(2, 2), stride=(2, 2))
    )
  )
  (conv6): Double_Conv(
    (double_conv): Sequential(
      (0): Conv2d(1024, 512, kernel_size=(3, 3), stride=(1, 1))
      (1): ReLU(inplace=True)
      (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1))
      (3): ReLU(inplace=True)
    )
  )
  (up_conv2): Up_Conv(
    (up_conv): Sequential(
```

```

        (0): ConvTranspose2d(512, 256, kernel_size=(2, 2), stride=(2, 2))
    )
)
(conv7): Double_Conv(
  (double_conv): Sequential(
    (0): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1))
    (3): ReLU(inplace=True)
  )
)
(up_conv3): Up_Conv(
  (up_conv): Sequential(
    (0): ConvTranspose2d(256, 128, kernel_size=(2, 2), stride=(2, 2))
  )
)
(conv8): Double_Conv(
  (double_conv): Sequential(
    (0): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1))
    (3): ReLU(inplace=True)
  )
)
(up_conv4): Up_Conv(
  (up_conv): Sequential(
    (0): ConvTranspose2d(128, 64, kernel_size=(2, 2), stride=(2, 2))
  )
)
(conv9): Double_Conv(
  (double_conv): Sequential(
    (0): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
    (3): ReLU(inplace=True)
  )
)
(conv10): Conv2d(64, 1, kernel_size=(1, 1), stride=(1, 1))
)
Epoch--[1/300], Loss: 0.24422259628772736
Epoch--[2/300], Loss: 0.17670384049415588
Epoch--[3/300], Loss: 0.16762495040893555
Epoch--[4/300], Loss: 0.15630751848220825
Epoch--[5/300], Loss: 0.1351277232170105
Epoch--[6/300], Loss: 0.11043376475572586
Epoch--[7/300], Loss: 0.10256646573543549
Epoch--[8/300], Loss: 0.09063447266817093
Epoch--[9/300], Loss: 0.08735042810440063
Epoch--[10/300], Loss: 0.08395235240459442
Epoch--[11/300], Loss: 0.08427760750055313
Epoch--[12/300], Loss: 0.08217491209506989
Epoch--[13/300], Loss: 0.08201637119054794
Epoch--[14/300], Loss: 0.0796237587928772
Epoch--[15/300], Loss: 0.07696548104286194
Epoch--[16/300], Loss: 0.0784609466791153
Epoch--[17/300], Loss: 0.07368180900812149
Epoch--[18/300], Loss: 0.07180894166231155
Epoch--[19/300], Loss: 0.07019085437059402
Epoch--[20/300], Loss: 0.06851570308208466
Epoch--[21/300], Loss: 0.07213474810123444

```

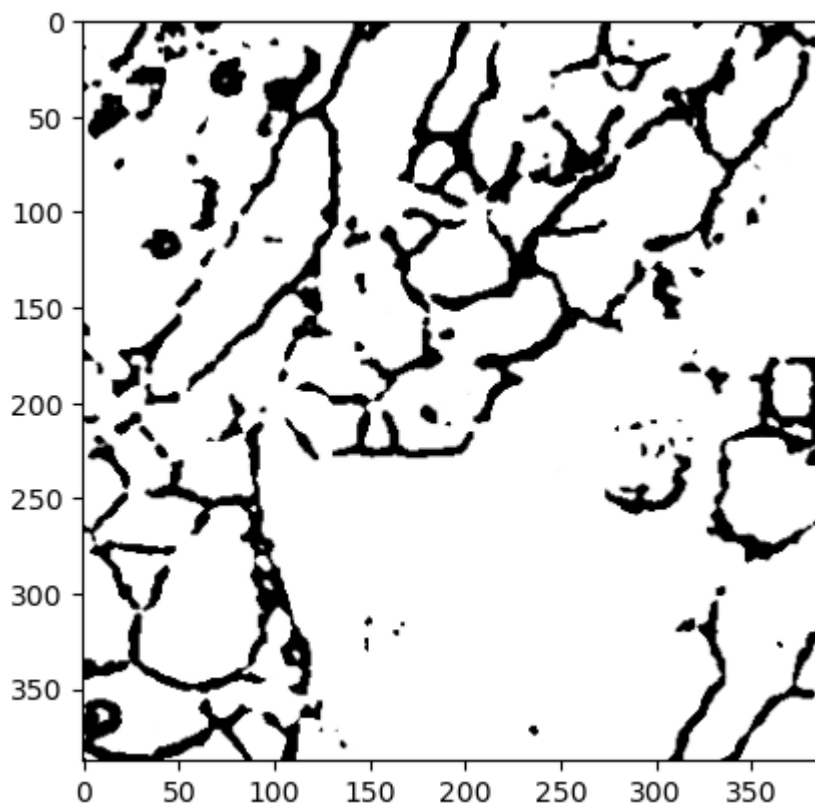
Epoch--[22/300], Loss: 0.06326540559530258
Epoch--[23/300], Loss: 0.060974497348070145
Epoch--[24/300], Loss: 0.059642255306243896
Epoch--[25/300], Loss: 0.05685649812221527
Epoch--[26/300], Loss: 0.05395294353365898
Epoch--[27/300], Loss: 0.05887637287378311
Epoch--[28/300], Loss: 0.05118934065103531
Epoch--[29/300], Loss: 0.04827766492962837
Epoch--[30/300], Loss: 0.04877633973956108
Epoch--[31/300], Loss: 0.046264056116342545
Epoch--[32/300], Loss: 0.04625805839896202
Epoch--[33/300], Loss: 0.04158412292599678
Epoch--[34/300], Loss: 0.03870035335421562
Epoch--[35/300], Loss: 0.03858955577015877
Epoch--[36/300], Loss: 0.034987933933734894
Epoch--[37/300], Loss: 0.032420288771390915
Epoch--[38/300], Loss: 0.03259747847914696
Epoch--[39/300], Loss: 0.029271777719259262
Epoch--[40/300], Loss: 0.027718273922801018
Epoch--[41/300], Loss: 0.02588723599910736
Epoch--[42/300], Loss: 0.026596752926707268
Epoch--[43/300], Loss: 0.023374078795313835
Epoch--[44/300], Loss: 0.022673513740301132
Epoch--[45/300], Loss: 0.02036995440721512
Epoch--[46/300], Loss: 0.021480632945895195
Epoch--[47/300], Loss: 0.02024017460644245
Epoch--[48/300], Loss: 0.01747109740972519
Epoch--[49/300], Loss: 0.018458688631653786
Epoch--[50/300], Loss: 0.01770872063934803
Epoch--[51/300], Loss: 0.015198790468275547
Epoch--[52/300], Loss: 0.014796853065490723
Epoch--[53/300], Loss: 0.014402016997337341
Epoch--[54/300], Loss: 0.014268873259425163
Epoch--[55/300], Loss: 0.013507016003131866
Epoch--[56/300], Loss: 0.012448279187083244
Epoch--[57/300], Loss: 0.012124099768698215
Epoch--[58/300], Loss: 0.01165382657200098
Epoch--[59/300], Loss: 0.010814868845045567
Epoch--[60/300], Loss: 0.011359408497810364
Epoch--[61/300], Loss: 0.010807271115481853
Epoch--[62/300], Loss: 0.010756919160485268
Epoch--[63/300], Loss: 0.010409530252218246
Epoch--[64/300], Loss: 0.009858166798949242
Epoch--[65/300], Loss: 0.009281644597649574
Epoch--[66/300], Loss: 0.009030894376337528
Epoch--[67/300], Loss: 0.008487960323691368
Epoch--[68/300], Loss: 0.008541123941540718
Epoch--[69/300], Loss: 0.008726521395146847
Epoch--[70/300], Loss: 0.008210175670683384
Epoch--[71/300], Loss: 0.008018391206860542
Epoch--[72/300], Loss: 0.007582376711070538
Epoch--[73/300], Loss: 0.009052516892552376
Epoch--[74/300], Loss: 0.007042134180665016
Epoch--[75/300], Loss: 0.007174388039857149
Epoch--[76/300], Loss: 0.007227578666061163
Epoch--[77/300], Loss: 0.006682974752038717
Epoch--[78/300], Loss: 0.006478009279817343
Epoch--[79/300], Loss: 0.006909225136041641
Epoch--[80/300], Loss: 0.006441338453441858
Epoch--[81/300], Loss: 0.006442085839807987

Epoch--[82/300], Loss: 0.006405854597687721
Epoch--[83/300], Loss: 0.005644606426358223
Epoch--[84/300], Loss: 0.00549681531265378
Epoch--[85/300], Loss: 0.0051025268621742725
Epoch--[86/300], Loss: 0.005194385070353746
Epoch--[87/300], Loss: 0.004935002885758877
Epoch--[88/300], Loss: 0.005399954039603472
Epoch--[89/300], Loss: 0.005475777667015791
Epoch--[90/300], Loss: 0.004414193797856569
Epoch--[91/300], Loss: 0.004215549677610397
Epoch--[92/300], Loss: 0.004921549465507269
Epoch--[93/300], Loss: 0.004936507903039455
Epoch--[94/300], Loss: 0.00525258481502533
Epoch--[95/300], Loss: 0.003908907528966665
Epoch--[96/300], Loss: 0.003845849772915244
Epoch--[97/300], Loss: 0.004068668000400066
Epoch--[98/300], Loss: 0.0034939053002744913
Epoch--[99/300], Loss: 0.0032513164915144444
Epoch--[100/300], Loss: 0.0032397478353232145
Epoch--[101/300], Loss: 0.006313052959740162
Epoch--[102/300], Loss: 0.0039518470875918865
Epoch--[103/300], Loss: 0.0032158507965505123
Epoch--[104/300], Loss: 0.0031867169309407473
Epoch--[105/300], Loss: 0.0033672181889414787
Epoch--[106/300], Loss: 0.0032220352441072464
Epoch--[107/300], Loss: 0.002874411176890135
Epoch--[108/300], Loss: 0.002583626890555024
Epoch--[109/300], Loss: 0.0028189842123538256
Epoch--[110/300], Loss: 0.0030703835655003786
Epoch--[111/300], Loss: 0.0026699122972786427
Epoch--[112/300], Loss: 0.002309014555066824
Epoch--[113/300], Loss: 0.002579868072643876
Epoch--[114/300], Loss: 0.0024014238733798265
Epoch--[115/300], Loss: 0.00225701997987926
Epoch--[116/300], Loss: 0.0022424268536269665
Epoch--[117/300], Loss: 0.0028746058233082294
Epoch--[118/300], Loss: 0.0024406197480857372
Epoch--[119/300], Loss: 0.002044287044554949
Epoch--[120/300], Loss: 0.0020936117507517338
Epoch--[121/300], Loss: 0.0023111801128834486
Epoch--[122/300], Loss: 0.0022491533309221268
Epoch--[123/300], Loss: 0.0025777199771255255
Epoch--[124/300], Loss: 0.0020581157878041267
Epoch--[125/300], Loss: 0.002004412468522787
Epoch--[126/300], Loss: 0.002323859604075551
Epoch--[127/300], Loss: 0.0025701040867716074
Epoch--[128/300], Loss: 0.0017881273524835706
Epoch--[129/300], Loss: 0.0018021801952272654
Epoch--[130/300], Loss: 0.0017408289713785052
Epoch--[131/300], Loss: 0.0018773669144138694
Epoch--[132/300], Loss: 0.0016981037333607674
Epoch--[133/300], Loss: 0.0018634959124028683
Epoch--[134/300], Loss: 0.0017280550673604012
Epoch--[135/300], Loss: 0.0016503001097589731
Epoch--[136/300], Loss: 0.0016210668254643679
Epoch--[137/300], Loss: 0.001604536664672196
Epoch--[138/300], Loss: 0.0015213359147310257
Epoch--[139/300], Loss: 0.0016333762323483825
Epoch--[140/300], Loss: 0.0016337529523298144
Epoch--[141/300], Loss: 0.0014155760873109102

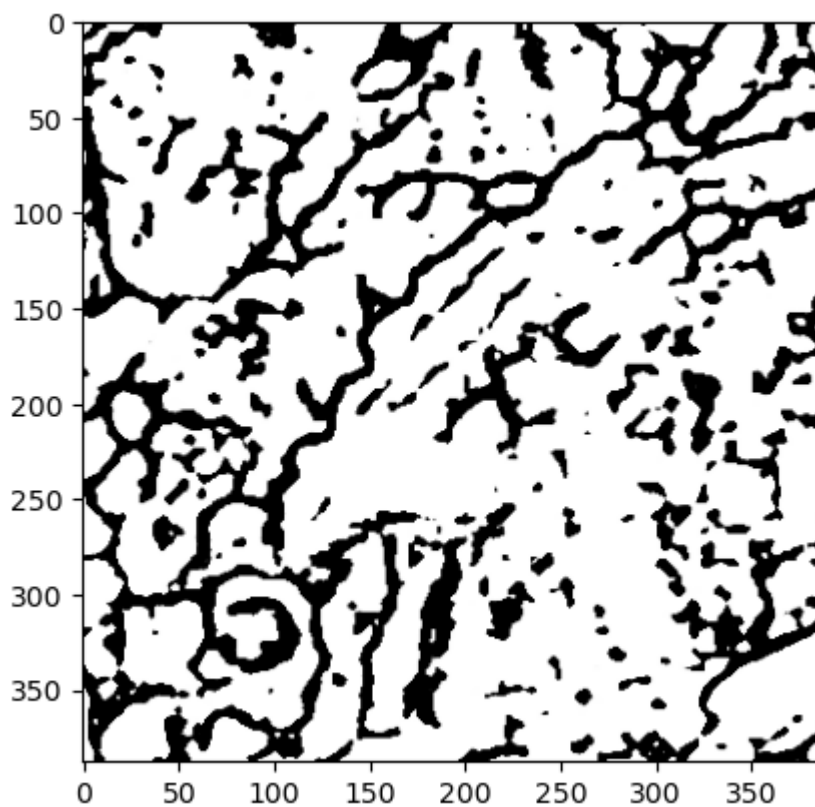
Epoch--[142/300], Loss: 0.0015869312919676304
Epoch--[143/300], Loss: 0.001866018632426858
Epoch--[144/300], Loss: 0.001606168458238244
Epoch--[145/300], Loss: 0.0016599257942289114
Epoch--[146/300], Loss: 0.001541893812827766
Epoch--[147/300], Loss: 0.0015678118215873837
Epoch--[148/300], Loss: 0.0016394441481679678
Epoch--[149/300], Loss: 0.0016375855775550008
Epoch--[150/300], Loss: 0.0015219635097309947
Epoch--[151/300], Loss: 0.00126122054643929
Epoch--[152/300], Loss: 0.0013864664360880852
Epoch--[153/300], Loss: 0.0014087375020608306
Epoch--[154/300], Loss: 0.0013570324517786503
Epoch--[155/300], Loss: 0.0014473397750407457
Epoch--[156/300], Loss: 0.0013970083091408014
Epoch--[157/300], Loss: 0.0013564806431531906
Epoch--[158/300], Loss: 0.0014553844230249524
Epoch--[159/300], Loss: 0.0014071919722482562
Epoch--[160/300], Loss: 0.00125631142873317
Epoch--[161/300], Loss: 0.0012061638990417123
Epoch--[162/300], Loss: 0.0012173980940133333
Epoch--[163/300], Loss: 0.0011632785899564624
Epoch--[164/300], Loss: 0.0012110413517802954
Epoch--[165/300], Loss: 0.001298957271501422
Epoch--[166/300], Loss: 0.0011859263759106398
Epoch--[167/300], Loss: 0.0011515270452946424
Epoch--[168/300], Loss: 0.0011472641490399837
Epoch--[169/300], Loss: 0.0012362226843833923
Epoch--[170/300], Loss: 0.0012324745766818523
Epoch--[171/300], Loss: 0.0013529915595427155
Epoch--[172/300], Loss: 0.0011269607348367572
Epoch--[173/300], Loss: 0.0012090326054021716
Epoch--[174/300], Loss: 0.0011631374945864081
Epoch--[175/300], Loss: 0.0011384160025045276
Epoch--[176/300], Loss: 0.0012395166559144855
Epoch--[177/300], Loss: 0.0011896558571606874
Epoch--[178/300], Loss: 0.0012149629183113575
Epoch--[179/300], Loss: 0.001178179052658379
Epoch--[180/300], Loss: 0.0011421868111938238
Epoch--[181/300], Loss: 0.009188042022287846
Epoch--[182/300], Loss: 0.003686075797304511
Epoch--[183/300], Loss: 0.001635481254197657
Epoch--[184/300], Loss: 0.0012838210677728057
Epoch--[185/300], Loss: 0.0011729615507647395
Epoch--[186/300], Loss: 0.0011291588889434934
Epoch--[187/300], Loss: 0.0011018997756764293
Epoch--[188/300], Loss: 0.0010876174783334136
Epoch--[189/300], Loss: 0.0011064144782721996
Epoch--[190/300], Loss: 0.0010462383506819606
Epoch--[191/300], Loss: 0.0010332678211852908
Epoch--[192/300], Loss: 0.001014849403873086
Epoch--[193/300], Loss: 0.0010343882022425532
Epoch--[194/300], Loss: 0.001017716946080327
Epoch--[195/300], Loss: 0.001035989378578961
Epoch--[196/300], Loss: 0.001015283865854144
Epoch--[197/300], Loss: 0.001030577695928514
Epoch--[198/300], Loss: 0.0010280291317030787
Epoch--[199/300], Loss: 0.0009986190125346184
Epoch--[200/300], Loss: 0.0010137753561139107
Epoch--[201/300], Loss: 0.0010186528088524938

Epoch--[202/300], Loss: 0.0010221238480880857
Epoch--[203/300], Loss: 0.0009657470509409904
Epoch--[204/300], Loss: 0.001056564855389297
Epoch--[205/300], Loss: 0.0009891451336443424
Epoch--[206/300], Loss: 0.0009924264159053564
Epoch--[207/300], Loss: 0.0010078599443659186
Epoch--[208/300], Loss: 0.0010071136057376862
Epoch--[209/300], Loss: 0.0009849006310105324
Epoch--[210/300], Loss: 0.0010628629243001342
Epoch--[211/300], Loss: 0.0010014328872784972
Epoch--[212/300], Loss: 0.000980717595666647
Epoch--[213/300], Loss: 0.0009602458449080586
Epoch--[214/300], Loss: 0.0009761596447788179
Epoch--[215/300], Loss: 0.001004837336950004
Epoch--[216/300], Loss: 0.0010086463298648596
Epoch--[217/300], Loss: 0.0010127094574272633
Epoch--[218/300], Loss: 0.0010313928360119462
Epoch--[219/300], Loss: 0.0010044390801340342
Epoch--[220/300], Loss: 0.001019151764921844
Epoch--[221/300], Loss: 0.0009862302104011178
Epoch--[222/300], Loss: 0.0010057679610326886
Epoch--[223/300], Loss: 0.0009956042049452662
Epoch--[224/300], Loss: 0.000991294626146555
Epoch--[225/300], Loss: 0.0009699888760223985
Epoch--[226/300], Loss: 0.000988326733931899
Epoch--[227/300], Loss: 0.0009584494982846081
Epoch--[228/300], Loss: 0.0009797255042940378
Epoch--[229/300], Loss: 0.0009841988794505596
Epoch--[230/300], Loss: 0.0009500873857177794
Epoch--[231/300], Loss: 0.0009632181609049439
Epoch--[232/300], Loss: 0.001020397525280714
Epoch--[233/300], Loss: 0.0009716380736790597
Epoch--[234/300], Loss: 0.0009465436451137066
Epoch--[235/300], Loss: 0.0009861791040748358
Epoch--[236/300], Loss: 0.0010219437535852194
Epoch--[237/300], Loss: 0.0010004362557083368
Epoch--[238/300], Loss: 0.0009889270877465606
Epoch--[239/300], Loss: 0.000969322572927922
Epoch--[240/300], Loss: 0.0009983066702261567
Epoch--[241/300], Loss: 0.001251843525096774
Epoch--[242/300], Loss: 0.0010941445361822844
Epoch--[243/300], Loss: 0.0010406688088551164
Epoch--[244/300], Loss: 0.0011142073199152946
Epoch--[245/300], Loss: 0.0010050119599327445
Epoch--[246/300], Loss: 0.0009862222941592336
Epoch--[247/300], Loss: 0.0009815300581976771
Epoch--[248/300], Loss: 0.0009886362822726369
Epoch--[249/300], Loss: 0.0010433169081807137
Epoch--[250/300], Loss: 0.0009796066442504525
Epoch--[251/300], Loss: 0.0009546466753818095
Epoch--[252/300], Loss: 0.0009962524054571986
Epoch--[253/300], Loss: 0.0010219765827059746
Epoch--[254/300], Loss: 0.0010032388381659985
Epoch--[255/300], Loss: 0.0010740087600424886
Epoch--[256/300], Loss: 0.0017299315659329295
Epoch--[257/300], Loss: 0.006545150652527809
Epoch--[258/300], Loss: 0.003648733487352729
Epoch--[259/300], Loss: 0.001891933847218752
Epoch--[260/300], Loss: 0.0014354718150570989
Epoch--[261/300], Loss: 0.0014735631411895156

Epoch--[262/300], Loss: 0.0010930591961368918
Epoch--[263/300], Loss: 0.0011756831081584096
Epoch--[264/300], Loss: 0.00099862867500633
Epoch--[265/300], Loss: 0.0009650186984799802
Epoch--[266/300], Loss: 0.0009303633705712855
Epoch--[267/300], Loss: 0.0009304494597017765
Epoch--[268/300], Loss: 0.0009112788829952478
Epoch--[269/300], Loss: 0.0009319978998973966
Epoch--[270/300], Loss: 0.001001371187157929
Epoch--[271/300], Loss: 0.0009120179456658661
Epoch--[272/300], Loss: 0.0008959760889410973
Epoch--[273/300], Loss: 0.00089462153846398
Epoch--[274/300], Loss: 0.0008997913100756705
Epoch--[275/300], Loss: 0.000885111978277564
Epoch--[276/300], Loss: 0.0009452239610254765
Epoch--[277/300], Loss: 0.0008977993857115507
Epoch--[278/300], Loss: 0.0008888422162272036
Epoch--[279/300], Loss: 0.0009076359565369785
Epoch--[280/300], Loss: 0.0008892943733371794
Epoch--[281/300], Loss: 0.0008820961811579764
Epoch--[282/300], Loss: 0.0008734849980100989
Epoch--[283/300], Loss: 0.0008792763110250235
Epoch--[284/300], Loss: 0.0008919027168303728
Epoch--[285/300], Loss: 0.0008743159705772996
Epoch--[286/300], Loss: 0.0008642725879326463
Epoch--[287/300], Loss: 0.0008696053409948945
Epoch--[288/300], Loss: 0.0008743295329622924
Epoch--[289/300], Loss: 0.0008817154448479414
Epoch--[290/300], Loss: 0.0008982231956906617
Epoch--[291/300], Loss: 0.0008749848930165172
Epoch--[292/300], Loss: 0.0008680042810738087
Epoch--[293/300], Loss: 0.0008621918386779726
Epoch--[294/300], Loss: 0.0008828852442093194
Epoch--[295/300], Loss: 0.0008558187982998788
Epoch--[296/300], Loss: 0.0008666281937621534
Epoch--[297/300], Loss: 0.0008483412093482912
Epoch--[298/300], Loss: 0.0008662038599140942
Epoch--[299/300], Loss: 0.0008723214268684387
Epoch--[300/300], Loss: 0.0008605000912211835
Finished Training
Saved PyTorch Model State to model.pth



```
In [4]: test_prediction = model_test(test_iter[7])  
img_grid = make_grid(test_prediction)  
  
# show images  
Matplotlib_Imshow(img_grid)
```



```
In [5]: print(loss.item())
```

0.0008048693416640162

In []: